

Portal de Ordenamiento Territorial

San Pedro Tlaquepaque

Revisión de integridad de código, capas de seguridad y control de calidad sobre la aplicación completa (frontend Remix 3 + backend Bun/Postgres). Todos los hallazgos listados aquí ya están corregidos y verificados en la rama de trabajo.

RAMA	COMMIT	FECHA	ALCANCE
claude	6d25597	24 ago. 2026	app/ + backend/

HALLAZGO CRÍTICO — CADENA COMPLETA

Cualquier visitante podía volverse admin y ver los datos de todos los ciudadanos

- 1. POST /api/auth/register aceptaba role directo del body, sin sesión previa. Bastaba enviar role: "admin" para crear una cuenta administradora — backend/src/app.ts.
- 2. Aun sin usar ese atajo, requireAdminUser() en app/backend.ts solo comprobaba "hay sesión", no el rol. Cualquier cuenta user creada desde el propio formulario público de /login ya entraba a /admin/*.
- 3. Desde ahí, los endpoints de participaciones (listado, detalle, adjuntos, export a Word, búsqueda) solo exigían requireAuth(), no requireAdmin(): nombre, correo, domicilio, observaciones y documentos de cada ciudadano quedaban a un registro de distancia.

Corregido en las tres capas: el registro público ya ignora el role enviado por el cliente, requireAdminUser() exige role === 'admin', y los endpoints con datos personales pasaron a requireAdmin(). Ver hallazgos S1–S3 más abajo.

01

Integridad de código y clean code

El proyecto pasó de un starter mínimo a una plataforma completa (portal ciudadano + panel admin + backend propio) sin que las herramientas de calidad se mantuvieran al día: typecheck y lint no llegaban a correr.

ALTO BUILD ROTO CORREGIDO

typescript-eslint no soportaba TypeScript 7.0

Ambas dependencias estaban fijadas a "latest"; TypeScript 7.0 se instaló antes de que typescript-eslint lo soportara (rango declarado: <6.1.0) y npm run lint no llegaba a arrancar. Se fija typescript a ~5.7 en el proyecto raíz.

package.json

MEDIO CORREGIDO

El carrusel del hero y el editor de fotos nunca se ejecutaban

Usaban `dangerouslySetInnerHTML`, una prop de React que este framework no implementa (el runtime SSR la serializa como atributo literal, no como contenido). El prop real es `innerHTML` en texto plano.

`app/actions/home-page.tsx` · `app/actions/admin/personalizacion-page.tsx`

MEDIO CORREGIDO

La personalización de marca no llegaba a la portada pública

Cuatro componentes de la home (Hero, "Qué es este sitio", tarjetas, footer) recibían `{ theme }` desestructurado directo en vez del patrón `handle: Handle<Props>` de este framework — `theme` llegaba `undefined` siempre, sin importar lo guardado en Personalización.

`app/actions/home-page.tsx`

BAJO CORREGIDO

El campo "Domicilio" se validaba y se perdía

El formulario público de participación lo pedía y lo validaba, pero el controller nunca lo incluía en el envío al backend. Ahora se reenvía como `calle`.

`app/actions/participation/controller.tsx`

BAJO CORREGIDO

"Hasta 850 MB" cuando el backend limita a 220 MB

Un archivo de más de 220 MB pasaba la expectativa visual del formulario y el backend lo rechazaba con un 413 confuso. Se corrigió el texto en portada y en el formulario para reflejar el límite real.

`app/actions/home-page.tsx` · `app/actions/participation/page.tsx`

BAJO CORREGIDO

Dos lockfiles para el mismo package.json

`bun.lock` (el que usa el Dockerfile) y `package-lock.json` convivían y podían resolver versiones distintas. Se elimina el segundo y se gitignora.

`package-lock.json` · `.gitignore`

BAJO CORREGIDO

38 errores de ESLint en el proyecto

Variables sin usar, `let` que debían ser `const`, y varios `any` explícitos. Donde el `any` era legítimo (JSONB dinámico de Postgres, un `deep-merge` genérico, la config de tema externa) se documentó con un comentario y un `eslint-disable` puntual en vez de forzar un tipo ficticio.

`app/` · `backend/src/`

CORREGIDO

Faltaba la ruta para borrar sesiones POEL

El botón "eliminar" del panel POEL enviaba `DELETE /api/poel/:id`, pero esa ruta no existía en el backend — el import de `deletePoelSesion` estaba muerto, justo la pista que llevó al bug.

`backend/src/app.ts`

Capas de seguridad

Procesos, código, APIs, base de datos, usuarios y rutas protegidas. Los tres primeros hallazgos son la cadena del recuadro de arriba, listados aquí con su corrección puntual.

CRITICO **CORREGIDO**

S1 — Autoregistro con rol de administrador

POST /api/auth/register es público por diseño (crea cuentas ciudadanas) pero confiaba en el role del body. Ahora se ignora y siempre se crea con role: 'user'; crear admins solo es posible ya autenticado como admin, vía POST /api/users.

backend/src/app.ts

CRITICO **CORREGIDO**

S2 — El guard de admin no comprobaba el rol

requireAdminUser() devolvía cualquier usuario con sesión válida. Lo usan las ocho rutas de /admin/*: un solo cambio en la función cierra el acceso en todas a la vez.

app/backend.ts

CRITICO **CORREGIDO**

S3 — PII de participaciones accesible sin ser admin

Listado, detalle, adjuntos, export a Word, envío por correo, búsqueda y stats solo exigían requireAuth(). Se suben a requireAdmin() como defensa en profundidad, independiente del fix en S2.

backend/src/app.ts

ALTO **CORREGIDO**

S4 — SESSION_SECRET podía quedar en su valor por defecto

Las cookies de sesión se firman con HMAC sobre SESSION_SECRET. Sin variable de entorno, caía en un placeholder público visible en el código fuente — cualquiera podía forjar una sesión de admin. A diferencia de ROOT_PASSWORD, no fallaba rápido. Ahora sí: falla en arranque si NODE_ENV=production y no está definida, y se quitó también el default débil de ambos docker-compose.yml.

backend/src/auth/auth.ts · docker-compose.yml

ALTO **CORREGIDO**

S5 — XSS almacenado vía colores del tema

Los colores guardados desde "Personalización y Marca" se interpolaban sin escapar dentro de un <script> (carrusel de la portada) y un <style> (panel admin). Combinado con S1, un registro público bastaba para inyectar JavaScript visible a todos los ciudadanos. Se agregó isSafeCssColor() — exige hex o rgb()/rgba() — antes de interpolar en cualquiera de los dos.

app/ui/civic-horizon.ts · app/actions/home-page.tsx · app/ui/admin/admin-layout.tsx

MEDIO **CORREGIDO**

S6 — Sin límite de intentos de login

Nada impedía fuerza bruta contra la cuenta ROOT (correo predecible: root@ordenamiento.gob.mx). Se agregó un limitador en memoria: 10 intentos fallidos por correo cada 15 minutos, suficiente para el despliegue actual de una sola instancia.

backend/src/auth/auth.ts

MEDIO CORREGIDO

S7 — Comparación de firma no era en tiempo constante

`verifySessionToken` comparaba la firma HMAC con `!==`. Se cambió a `crypto.timingSafeEqual` para no filtrar información por el tiempo de respuesta.

`backend/src/auth/auth.ts`

MEDIO CORREGIDO

S8 — "Correo no configurado" nunca se mostraba

`mailConfigurado()` comprobaba una constante con fallback a `127.0.0.1`, así que siempre daba `true`. Sin SMTP real, el sistema intentaba conectar a `localhost` y fallaba de forma confusa en vez de responder el 503 esperado. Ahora comprueba `process.env.SMTP_HOST` directo.

`backend/src/services/mail.ts`

BAJO CORREGIDO

S9 — Correo personal hardcodeado como destino de prueba

El endpoint de correo de prueba caía a una dirección de Gmail personal si no se enviaba para. Se quitó el default; ahora el campo es obligatorio.

`backend/src/app.ts`

BAJO CORREGIDO

S10 — Lista blanca de tablas solo existía en tipos

`getIdfMap` interpola un nombre de tabla como identificador SQL, protegido solo por un tipo `IdCorpus` que se borra al compilar. Hoy nunca recibe entrada de usuario, pero se agregó también la validación en runtime para que un futuro llamador no abra una inyección por accidente.

`backend/src/services/knowledge.ts`

BAJO CORREGIDO

S11 — Postgres y la API expuestos en el compose de desarrollo

`backend/docker-compose.yml` (uso local del backend por separado) publicaba 5432 y 5920 en todas las interfaces con la contraseña por defecto de Postgres. Se ligan a `127.0.0.1`, igual que ya hacía el compose de la raíz.

`backend/docker-compose.yml`

03

QA y verificación

No existía ni un solo archivo de test pese a tener `vitest` y `bun:test` configurados. Se agregaron pruebas para la lógica que este trabajo tocó — sobre todo la de seguridad — y se corrió el ciclo completo hasta quedar en verde.

CHEQUEO	RAÍZ (app/)	BACKEND
Typecheck	Verde	Verde
Lint (ESLint)	Verde · 0/38	— sin config —
Formato (Prettier)	Verde	— sin config —

Tests	6/6	25/25
-------	-----	-------

Tests nuevos

- `app/ui/civic-horizon.test.ts` — `isSafeCssColor`, incluyendo los intentos de inyección concretos que motivaron el fix de S5.
- `backend/src/auth/auth.test.ts` — firma y verificación de sesión, tampering, expiración a los 7 días, y el rate limiter de S6.
- `backend/src/vector/tfidf.test.ts` y `backend/src/text/chunk.test.ts` — la matemática pura del motor de búsqueda semántica.

Verificación manual en navegador (*dev server, sin Postgres/backend disponibles en este entorno*):

- Home, login, participación y poetdum renderizan completas, sin errores de consola.
- El mapa Leaflet de poetdum carga correctamente tras mover su CSS del `<Document>` global a esa única página.
- `/admin/*` responde 500 sin filtrar detalle alguno — comportamiento esperado sin backend, no una regresión introducida aquí.

04

Fuera de alcance de este pase

Con riesgo/beneficio en contra de tocarlas sin batería de pruebas ni entorno con base de datos disponible:

- Reescribir `personalizacion-page.tsx` a componentes hidratados con `clientEntry/on()` — hoy funciona correctamente como HTML estático; el cambio sería puramente estilístico y de alto riesgo sin cobertura de tests.
- Subresource Integrity en los scripts de terceros (Google Fonts, Leaflet, Iconify) que carga Document — mitigaría un CDN comprometido, pero requiere fijar y verificar hashes por versión.
- Validar la forma completa del tema en el límite de escritura (`saveCustomizations`), no solo al leerlo — hoy la defensa contra inyección vive en el punto de render, que ya es suficiente por sí sola.